Federated Fine-Tuning of a Vision-Language Decision Model for UAV Power-Line Inspection

Mahdi Khashan

[a][Affiliation to be added],

Abstract

Onboard vision-language models let inspection drones describe what they see, but generative models need seconds per video segment on embedded hardware, and the images that would improve them stay with the fleet that collected them. We study a vision-language decision model, laya-vision (201M parameters), which answers typed multiple-choice and yes/no questions about an image in one forward pass without generating text. On 35,415 power-line component images from ElecVQA-Bench (InsPLAD), the published checkpoint fails zero-shot: its condition macro F1 is 0.180, close to the 0.158 of always answering "good". Fine-tuning on 4,807 class-balanced images raises the condition macro F1 to 0.703, component accuracy to 0.924, defect AUROC to 0.965, and item accuracy to 0.948, close to a fine-tuned ResNet-50 (0.961) and a LoRA-tuned SmolVLM-256M (0.956). Inference stays at 74 ms per image on an RTX 2080 Ti (0.25 s per 4 s segment). We then split the same images over simulated drones by tower and train with FedAvg under a fixed compute budget. A one-drone control matches central training, but accuracy falls as drones are added (macro F1 0.624, 0.541 and 0.474 for 3, 6 and 18 drones), while FedAvg with 6 drones still beats the average drone that trains alone (0.371). Starting the fleet from a model fine-tuned on separate server towers lifts fleet FedAvg from 0.513 to 0.636 macro F1 and from 0.932 to 0.953 defect AUROC, close to central training on all data. Full-model updates cost 6.9–41.3 GB of uplink per run.

Keywords: federated learning, vision-language model, decision model, UAV inspection, edge intelligence, power lines

1. Introduction

Utilities inspect power lines with unmanned aerial vehicles (UAVs) that record video for later human review. Moving the interpretation onboard reduces the data that leaves the drone. EVLM [1] runs a LoRA-tuned SmolVLM-256M [3] on an NVIDIA Jetson, filters key frames, and reduces the uplink of a 4 s segment from 485 kB to 25 kB. Its cost is latency: generating text takes 6.448 s per 4 s segment for a mission intent, and 1.4–2.3 s for the other intents in its evaluation [1]. A companion study profiles cloud, onboard and split deployment of such models for drones [2].

Many inspection questions do not need generated text. "Does this clamp have a defect?" and "Which condition does this insulator have?" are choices among known options. Decision models answer such typed questions in a single forward pass and return a calibrated probability for each option. Laya [4] introduced this design for text, and laya-vision [5] replaces its text encoder with a truncated SmolVLM backbone so that the state can be an image.

A fleet of drones collects the images such a model needs, but the images are often not shared. Different operators may own the drones, and imagery of critical infrastructure is sensitive. Federated learning [8] trains one model across clients without moving their data. The data of inspection drones differs strongly between clients: each drone flies its own towers, so its components and defect rates differ.

This paper asks three questions:

1. Can a small vision-language decision model, given only the image, classify power-line components and their condition well enough to serve as an onboard tool, and at what latency?
2. How close does federated fine-tuning across drones come to central fine-tuning when each drone holds the images of its own towers?
3. Does a centrally fine-tuned starting model help the federation?

Our contributions are:

- An evaluation of laya-vision on 35,415 InsPLAD component images, from zero-shot to two rounds of fine-tuning, with ground truth from the dataset folders, bootstrap intervals, a class-share correction, and item-level comparison with fine-tuned classifier and VLM tools.
- A latency measurement per image and per segment, placed next to the EVLM figures.

Email address: mahdikhashan1@gmail.com (Mahdi Khashan)

- A federated study with towers as drones: a one-drone control, FedAvg with 3, 6 and 18 drones, local-only training, and a comparison of a base start with a fine-tuned start on the same fleet.

2. Related work

Edge VLMs for UAV inspection. EVLM [1] and the cloud, edge or split study [2] are the closest systems. Both use a generative SmolVLM-256M and report text outputs per segment. INAR-VL [6] routes vision-language requests between edge and cloud with accuracy-aware policies. We study a non-generative model that answers fixed questions. Its outputs are labels and probabilities, not descriptions.

Decision models. Laya [4] scores typed questions (choice, yes/no, score) in one pass with a reinforcement-style objective and temperature calibration [7]. Laya-vision [5] adds image input. We use its published checkpoint and its fine-tuning recipe.

Federated learning. FedAvg [8] averages client models weighted by data size. Non-IID client data degrades FedAvg [9], and FedProx [10] adds a proximal term for heterogeneous clients. Flower [11] is a common framework for such simulations. Federated learning has been applied to the routing of language models [12] and to the uncertainty thresholds that decide when an edge model escalates to the cloud [13]. A recent study combines escalation of uncertain detections with federated adapter updates for guardrail inspection [14]. To our knowledge, federated fine-tuning of a vision-language decision model on drone inspection data has not been reported.

3. Method

3.1. Model and questions

Laya-vision [5] (checkpoint thaitea/laya-vision, revision f2fe3c1, 201M parameters) uses SmolVLM-256M cut to 20 of its 30 language layers, followed by a decision head. For a state and a typed question, the head returns one logit per option. A temperature per question type converts the logits to probabilities. The state in all our experiments is the image only. One predict call per image asks four questions:

- component (choice, 11 options): which power-line component is in the image.
- defect (yes/no): whether the component has a defect.
- condition (choice, 6 options): good, rust, corrosion, missing cap, bird nest, or other defect (torn part or peeling paint).
- judgeable (yes/no): whether the image is clear enough to judge. This question has no ground truth.

The image is encoded once and reused for the four questions.

3.2. Data and ground truth

ElecVQA-Bench [15] builds visual questions on the InsPLAD component crops [16]. We use its tower14 split: 18 training, 10 validation and 14 test towers, with 39,228 items on 35,415 unique images. The towers in different splits share no images. Each image carries its component folder and its class folder. We take the component (17 folders, with the 7 shackle types merged into one, giving 11 classes) and the condition (6 classes, with torn-up and peeling-paint merged) from these folders. A defect is any condition other than good. The folder labels agree with every ElecVQA-Bench answer. The test towers hold 6,092 images, of which 9.9% show a defect.

3.3. Fine-tuning

We follow the fine-tuning guide of laya-vision [5]. Each image gives three labelled questions (component, defect, condition). Training uses the RLCD objective of Laya with soft cross-entropy, AdamW, batch 8, learning rates 10^{-4} for the head and 2×10^{-5} for the backbone, 25 warm-up steps and 2.5 passes over the training questions. Temperatures are refit on held-out training images after training [7]. The RTX 2080 Ti has no native bfloat16, so all training runs in fp32. We train three models:

- Round 1: up to 400 images per condition (1,893 images, 79% defects; 1,793 for training), training the head and the last 4 language layers (22.6M parameters).
- Round 2, last 4 layers: all 2,249 defect images of the training towers plus up to 300 good images per component (5,107 images, 44% defects; 4,807 for training, 300 for temperatures).
- Round 2, full: the same data, training all parameters except the vision encoder (114.7M parameters).

3.4. Federated fine-tuning

The simulated drones use the same 5,107 images as round 2, divided by tower. For K drones, towers are assigned to drones so that the image counts are balanced. With $K = 18$ each drone holds one tower, from 1 to 1,103 images, and four drones hold no defect image. In round r, the server sends the global weights θ^r to every drone. Drone k trains on its n_k questions for 0.25 passes with a new AdamW optimizer and returns θ_k^r. The server averages

$$\theta^{r+1} = \sum_{k=1}^{K} \frac{n_k}{n} \theta_k^r, \qquad n = \sum_k n_k. \tag{1}$$

Ten rounds give 2.5 passes in total, so every run uses the same number of training questions as central training. The learning rate follows a cosine decrease over the rounds. The trained part is the full model of round 2. Each drone keeps 6% of its images out of training. After the last round the drones send the logits and labels of these images, and

the server fits the temperatures. No image leaves a drone. The simulation runs the drones one after another on one GPU, and the averaging in Eq. (1) is identical to the FedAvg strategy of Flower [11].

We run four kinds of experiment:

- Control: the same loop with one drone holding all images. It separates the effect of the loop from the effect of federation.
- FedAvg with $K = 3$, 6 and 18 drones.
- Local only: the six drones of $K = 6$ train alone for 2.5 passes each, with no averaging.
- Two starts: the 18 training towers are divided into a server group (13 towers, 2,554 images, 37% defects) and a fleet group (5 towers, 2,553 images, 51% defects), each with all components and conditions. A model is fine-tuned centrally on the server group. Then FedAvg with 5 fleet drones, one per tower, starts either from the published checkpoint or from the server model.

3.5. Evaluation

All numbers are on the test towers. For component and condition we report accuracy, balanced accuracy and macro F1, because the majority class covers 90% of the test images. For defect we report AUROC, average precision and expected calibration error (ECE). We tune a threshold on the validation towers that minimizes a risk of 10 per missed defect and 1 per false alarm, and report this risk per 1,000 images. Intervals are 95% bootstrap intervals over 1,000 resamples of the test images. Each configuration has one training seed.

Training sets with a different class mix than the test data shift the probabilities. As a correction that needs no retraining [17], we also report

$$\tilde{p}(c \mid x) \propto p(c \mid x)\, \frac{\pi_{\text{val}}(c)}{\pi_{\text{train}}(c)}, \tag{2}$$

where π are the label shares of the validation towers and of the training images. For comparison with earlier tools on the same benchmark, we also score the 7,057 test items of ElecVQA-Bench. A yes/no item takes $P(\text{defect}) \geq 0.5$, and a multiple-choice item takes the best condition among its options. The reference tools were evaluated on the same items. They are a ResNet-50 [18] classifier fine-tuned on the training towers, SmolVLM-256M tuned with LoRA [19] as in EVLM, and the same SmolVLM without tuning.

4. Results

4.1. Zero-shot

Without fine-tuning, laya-vision does not classify this data. Its condition accuracy is 0.834, below the 0.901 of always answering good. Its macro F1 is 0.180 (95% interval 0.172–0.190), against 0.158 for the majority answer. It calls 480 of the 601 defect images good, and the recall of corrosion, missing cap and bird nest is at most 0.02. The component accuracy is 0.141, below the majority baseline of 0.314. The defect question carries a weak signal (AUROC 0.714), with probabilities that are not calibrated (ECE 0.178). The judgeable answer does not predict a correct condition answer (AUROC 0.53).

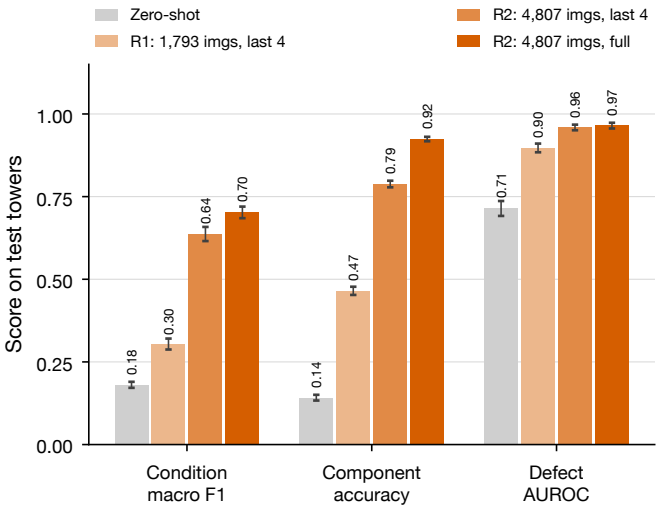


Figure 1: More data and a fully trained backbone lift every metric. Test-tower scores of laya-vision from zero-shot to the round-2 full fine-tune (6,092 images). Whiskers are 95% bootstrap intervals over images; one training seed per model; no class-share correction.

4.2. Fine-tuning

Figure 1 and Table 1 show the progression. Round 1 lifts the defect AUROC to 0.898. Its plain accuracy falls because the model learned a training mix with 79% defects and labels 1,971 of 5,491 good test images as defective. The correction in Eq. (2) raises its condition accuracy from 0.642 to 0.856 and its macro F1 from 0.303 to 0.451. Round 2 adds data, covers all components and has 44% defects. It improves every metric, and the full model is the best: condition accuracy 0.952 (above the majority answer), macro F1 0.703, component accuracy 0.924, defect AUROC 0.965 and ECE 0.058. For this model the correction changes little (condition accuracy 0.957). Its largest error is 152 good images called rust. Bird nests are always found, and the 5 test images of other defects are always missed. The spacer is the only component it does not recognize (recall 0.06, with 18 training images).

On the ElecVQA-Bench items the full model reaches 0.948, below the fine-tuned ResNet-50 (0.961) and SmolVLM with LoRA (0.956). The full model trains for 24 minutes on an RTX 2080 Ti with a peak memory of 3.6 GB. In training, most batches after step 1,600 reach the maximum reward, and the refit temperatures are large (4.1 for choice questions and 6.2 for yes/no questions). The test

Table 1: Test-tower results. Images: 6,092 test images. Items: 7,057 ElecVQA-Bench test items. The intervals for laya-vision are in the text and in Fig. 1. Reference tools are scored on the same items.

Model	Cond. macro F1	Comp. acc.	Defect AUROC	Item acc.
Zero-shot	0.180	0.141	0.714	0.830
Round 1, last 4	0.303	0.465	0.898	0.702
Round 2, last 4	0.637	0.789	0.959	0.923
Round 2, full	0.703	0.924	0.965	0.948
ResNet-50, tuned	–	–	–	0.961
SmolVLM + LoRA	–	–	–	0.956
SmolVLM, no tuning	–	–	–	0.830

results remain close to the validation results (condition accuracy 0.952 against 0.935, defect AUROC 0.965 against 0.968), so the model does not overfit the training towers.

4.3. Latency

On the RTX 2080 Ti in fp32, one predict call with all four questions takes a median of 72.0 ms per image (p95 98.5 ms), including reading and preprocessing the image. The fine-tuned full model takes 74.0 ms. The condition question alone takes 71.4 ms, so almost all of the time is spent encoding the image. Peak GPU memory for inference is 0.9 GB. With 3.45 key frames per 4 s segment, as in EVLM [1], one segment needs 0.25 s. On one thread of the server CPU, one image takes 2.5 s.

We have no Jetson measurement. We estimated a factor of 9 between a Quadro GV100 and a Jetson Orin NX from their fp32 throughput and memory bandwidth. The RTX 2080 Ti has a similar fp32 throughput to the GV100, so the same factor gives about 2.2 s per segment; this is an estimate. EVLM reports 1.4–2.3 s per segment for its non-description intents and 6.448 s for its mission intent on a Jetson [1]. The comparison is approximate: the hardware differs, EVLM generates text while laya-vision returns labels, and EVLM processes video segments while we process single crops.

4.4. Federated fine-tuning across drones

The control with one drone matches central training: macro F1 0.697 against 0.703 (Table 2). The intervals overlap for all three metrics, so the training loop does not cause the differences below. With the same data spread over more drones, FedAvg loses accuracy (Fig. 3b, c). From 1 to 18 drones, the condition macro F1 falls from 0.697 to 0.474 and the component accuracy from 0.933 to 0.700. The defect AUROC degrades less, from 0.959 to 0.933. The validation curves (Fig. 3a) show that the 6- and 18-drone runs are still improving after ten rounds, at 0.839 and 0.826, while the control has levelled off at 0.938. Every run receives the same total training, and the server averages updates that each cover only a part of the data. We therefore expect, but have not tested, that more rounds would close part of the gap.

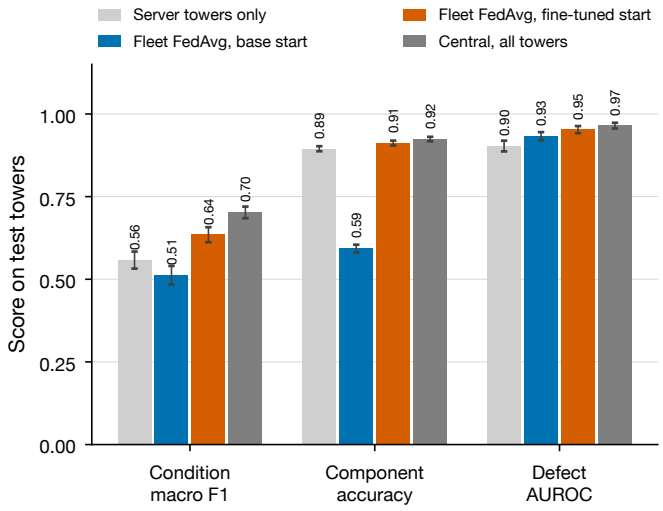


Figure 2: A fine-tuned start makes fleet FedAvg close to central training. The server model is trained on 13 towers. The fleet has 5 towers, one drone each, and starts either from the published checkpoint (base) or from the server model. Central training uses all 18 towers. Whiskers are 95% bootstrap intervals; one seed per run.

Federation still helps the individual drone. With six drones, FedAvg is better than the average drone that trains alone on every metric: macro F1 0.541 against 0.371, component accuracy 0.761 against 0.621, defect AUROC 0.939 against 0.881. It is also better than the best single drone in defect AUROC (0.939 against 0.916). The best single drone has a macro F1 of 0.525, inside the FedAvg interval, and a slightly higher component accuracy (0.782 against 0.761). Drones with few defects do worst alone: a drone with 29% defects reaches a macro F1 of 0.278.

4.5. A fine-tuned start for the fleet

Figure 2 compares the two starts on the same fleet. From the published checkpoint, fleet FedAvg reaches a macro F1 of 0.513, a component accuracy of 0.593 and a defect AUROC of 0.932. From the server model it reaches 0.636, 0.912 and 0.953, and the intervals of the two starts do not overlap. The fleet also improves the server model, from 0.558 to 0.636 macro F1 and from 0.903 to 0.953 AUROC, while its images stay on the drones. The fine-tuned start is already at 0.904 validation accuracy before the first round and ends at 0.939. Its item accuracy (0.937) is close to that of central training on all 18 towers (0.948).

4.6. Communication

The full-model update is 229.5 MB per drone and round in fp16, so a run needs 6.9 GB (3 drones) to 41.3 GB (18 drones) of uplink in total. Uploading all 5,107 training images once at about 35 kB each would take about 0.18 GB. Federation therefore does not save bandwidth in this setting. Its benefit is that the images stay on the drones. The updates are too large for the 4G uplink during a mission,

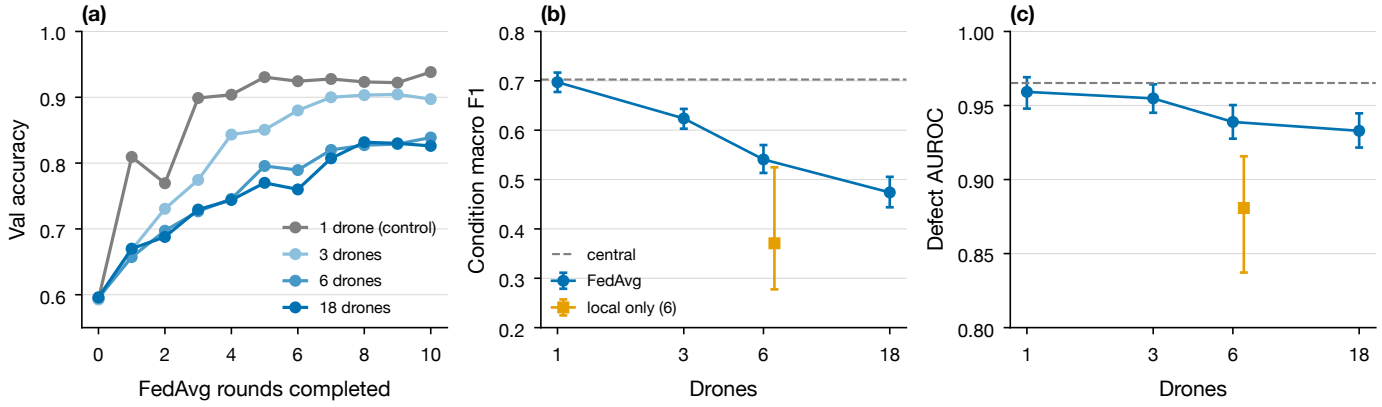


Figure 3: Splitting the same data over more drones slows FedAvg. (a) Validation accuracy on 600 images after each round. (b, c) Test-tower condition macro F1 and defect AUROC against the number of drones. FedAvg whiskers are 95% bootstrap intervals. The local-only marker is the mean of six drone models, and its whisker spans their minimum to maximum. The dashed line is central training. Every run uses the same total training (2.5 passes), with one seed per run.

Table 2: Federated runs on the test towers, with 95% bootstrap intervals. Local only: mean (minimum–maximum) of the six drone models. Uplink: full-model update in fp16 (229.5 MB) × drones × 10 rounds, calculated, not measured.

Run	Cond. macro F1 (95% interval)	Comp. acc.	Defect AUROC	Uplink
Central (round 2, full)	0.703 (0.685–0.720)	0.924	0.965	–
Control, 1 drone	0.697 (0.677–0.717)	0.933	0.959	–
FedAvg, 3 drones	0.624 (0.603–0.643)	0.874	0.955	6.9 GB
FedAvg, 6 drones	0.541 (0.513–0.570)	0.761	0.939	13.8 GB
FedAvg, 18 drones	0.474 (0.444–0.505)	0.700	0.933	41.3 GB
Local only, 6 drones	0.371 (0.278–0.525)	0.621	0.881	–
Server towers, central	0.558 (0.532–0.584)	0.894	0.903	–
Fleet FedAvg, base start	0.513 (0.484–0.540)	0.593	0.932	11.5 GB
Fleet FedAvg, fine-tuned start	0.636 (0.612–0.657)	0.912	0.953	11.5 GB

so they would be sent from the ground station after a mission. Sending only the head and the last four layers would cut the update to 45 MB, which we have not yet tested.

5. Discussion and limitations

A one-pass decision model is a practical onboard tool for this task once it is fine-tuned. It answers four questions in about the time a single question takes, and its estimated Jetson time per segment is at the top of EVLM's range for comparable intents. Zero-shot, it is not useful on this domain. Federated fine-tuning works, but under a fixed compute budget it pays for the spread of the data. The strongest result for a deployment is the combination of a centrally fine-tuned start and fleet FedAvg, which comes close to pooling all images.

The study has limits. Each configuration has one seed. The drones are simulated on one GPU, so update sizes and uplink are calculated, not measured. The latency comparison with EVLM relies on an estimated hardware factor and compares different outputs. The fixed compute budget is fair for compute but gives FedAvg with many drones fewer effective steps. The data comes from one benchmark with few images in some classes (5 test images of other defects, 17 spacers). The tower split groups images by location, but the drones are not real flights.

Next steps are longer FedAvg runs with fixed local steps, smaller updates (head and last layers), FedProx [10] for drones without defect images, more seeds, and a measurement on a Jetson.

6. Conclusion

Laya-vision, a 201M-parameter vision-language decision model, fails zero-shot on power-line component images. After fine-tuning on 4,807 images it reaches 0.948 item accuracy, close to fine-tuned ResNet-50 and SmolVLM tools, at 74 ms per image for four questions. FedAvg across tower-wise drones beats training alone but falls behind central training as the data is split further under a fixed budget. A model fine-tuned on separate server towers is the best starting point for the fleet, and with it federated training comes close to central training while the images stay on the drones.

Data and code availability

The code, the per-image predictions and the evaluation files are in the project repository, branch vision-fl (to

be made public on acceptance). ElecVQA-Bench and InsPLAD are public [15, 16]. The laya-vision weights are published under CC BY-NC-SA 4.0 [5].

Declaration of competing interest

The author declares no competing interests.

CRediT authorship contribution statement

Mahdi Khashan: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft, Writing – review & editing.

Declaration of generative AI use

During the preparation of this work the author used an AI assistant to write code, analyse results and draft text. The author reviewed and edited the content and takes full responsibility for the publication.

Acknowledgments

Experiments in this work were run and tracked with OpenResearch [20].

References

[1] R. Farahani, Z. Azimi, I. Murturi, A. Goknil, S. Sen, C. Timmerer, S. Dustdar, EVLM: Intent-driven edge vision language model for UAV-based power line inspection, in: 2026 IEEE International Conference on Edge Computing and Communications (EDGE), 2026. https://doi.org/10.1109/EDGE72783.2026.00014.

[2] Z. Azimi, R. Farahani, S. Dustdar, C. Timmerer, Cloud, edge, or split? Profiling onboard and split vision-language model deployment for drone AI, arXiv:2609.25415 (2026).

[3] A. Marafioti, et al., SmolVLM: Redefining small and efficient multimodal models, arXiv:2504.05299 (2025).

[4] Convai Innovations, Laya, software (2026). https://github.com/NandhaKishorM/laya.

[5] Laya Vision, software, commit 9e1e241 (2026). https://github.com/r33drichards/laya-vision; weights https://huggingface.co/thaitea/laya-vision.

[6] Sabanovic, Maliakel, Brandic, INAR-VL, arXiv:2605.18853 (2026).

[7] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: Proceedings of the 34th International Conference on Machine Learning (ICML), 2017.

[8] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), 2017.

[9] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, V. Chandra, Federated learning with non-IID data, arXiv:1806.00582 (2018).

[10] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith, Federated optimization in heterogeneous networks, in: Proceedings of Machine Learning and Systems (MLSys), 2020.

[11] D. J. Beutel, et al., Flower: A friendly federated learning research framework, arXiv:2007.14390 (2020).

[12] Askin, et al., Federate the router, arXiv:2601.22318 (2026).

[13] Solat, Lee, Seif, Niyato, Poor, FedHLM, IEEE Internet of Things Journal (2025). https://doi.org/10.1109/jiot.2025.3618095.

[14] From open-vocabulary detection to structural assessment: An agentic federated approach for guardrail monitoring (2026). https://www.sciencedirect.com/science/article/pii/S2772503026000459.

[15] ElecVQA-Bench, dataset and code repository (2026). https://github.com/GhostDino/elecvqa-bench.

[16] A. L. B. Vieira e Silva, et al., InsPLAD: A dataset and benchmark for power line asset inspection in UAV images, International Journal of Remote Sensing (2023). arXiv:2311.01619.

[17] M. Saerens, P. Latinne, C. Decaestecker, Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure, Neural Computation 14 (1) (2002) 21–41.

[18] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[19] E. J. Hu, et al., LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR), 2022.

[20] alphaXiv, OpenResearch: The local-first harness and workspace for research agents (2026). https://github.com/alphaXiv/OpenResearch.